



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0402-OPERATING SYSTEMS

BUILDING A CONCURRENT WEB SERVER SIMULATION: A THREAD-BASED APPROACH TO REQUEST HANDLING AND SYNCHRONIZATION CONTROL

Guided by:

Dr. Priskilla Angel Rani J

Presented by

Sirigiri Uma (192525054)

Rithika G(192571027)

Abbisetty Madhuri(192525377)

Baba Satya Kumar K(192525226)

Prawin Sankar H(192524002)



ENGINEER TO EXCEL



AGENDA



- ❖ Abstract
- ❖ Introduction
- ❖ Problem Identification and Analysis
- ❖ Solution Design and Implementation
- ❖ Module 1: Client Request Handling Module
- ❖ Module 2: Thread Management Module
- ❖ Module 3: Concurrency Control Module
- ❖ Module 4: Response Generation Module
- ❖ Module 5: Logging and Monitoring Module
- ❖ Results and Recommendations
- ❖ Reflection on Learning and Personal Development
- ❖ Conclusion

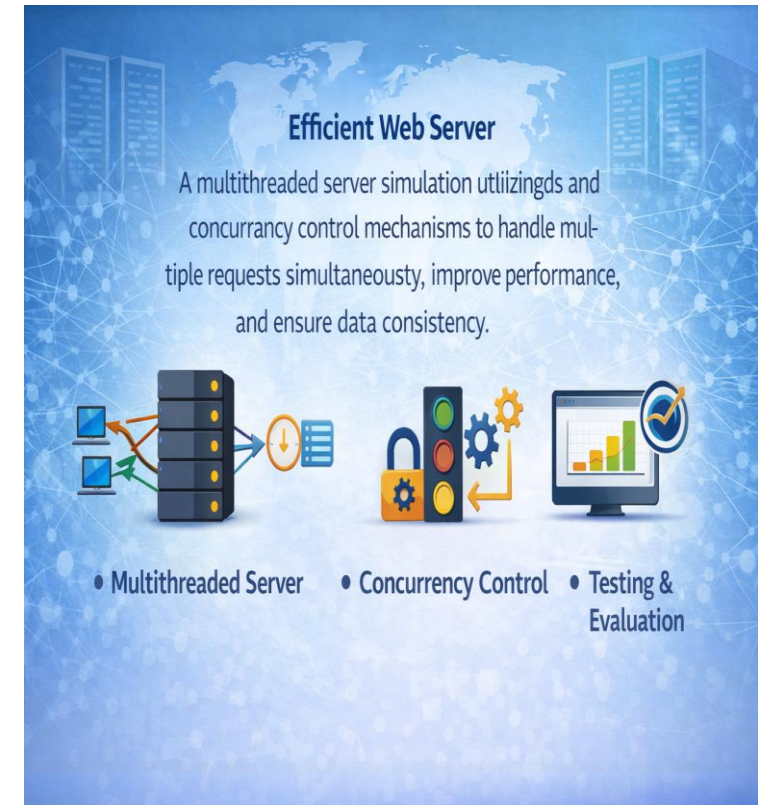


ENGINEER TO EXCEL

Abstract



- Simulates a concurrent web server using a thread-based architecture.
- Uses multiple threads with synchronization controls.
- Serves multiple client requests simultaneously.
- Delivers higher throughput than single-threaded servers.
- Assigns a dedicated thread to each request.
- Reduces wait time and speeds up processing.
- Applies mutexes and semaphores for synchronization.
- Prevents race conditions and deadlocks.
- Maintains data consistency and system stability.
- Tested under varied loads for efficiency and reliability.
- Confirms the thread-based approach improves server performance



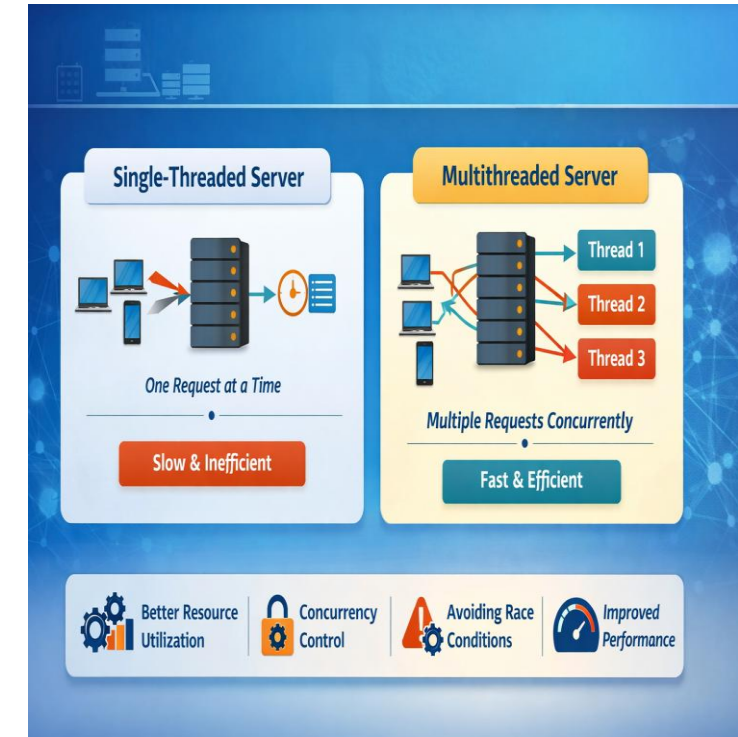


ENGINEER TO EXCEL

Introduction



- A web server accepts requests and returns responses.
- Single-threaded servers process only one request at a time.
- Causes delays with multiple simultaneous users.
- A faster, more efficient approach is needed.
- Thread-based design handles many requests in parallel.
- Boosts performance, responsiveness, and resource use.
- Requires synchronization to protect shared resources.
- Supports real-time and high-traffic applications.
- Reduces CPU idle time and boosts efficiency.
- Forms the basis for modern high-performance servers.





ENGINEER TO EXCEL

Problem Identification and Analysis



- A single-threaded server handles only one request at a time.
- More users mean longer client wait times.
- Performance and response times suffer under heavy load.
- System resources go underused, with long CPU idle periods.
- Struggles to serve multiple clients simultaneously.
- Shared-data access risks race conditions.
- Concurrent execution risks deadlocks.
- Weak synchronization causes data inconsistency.
- A truly concurrent processing system is required.
- Safe execution needs dedicated concurrency control.



ENGINEER TO EXCEL

Solution Design and Implementation



- Design a thread-based web server architecture.
- Use a thread pool to serve requests efficiently.
- Use a request queue to organize incoming requests.
- Route each request to its own thread.
- Apply synchronization via mutex and semaphore.
- Control access to shared resources safely.
- Add concurrency control to avoid race conditions and deadlocks.
- Optimize resource utilization for performance.
- Implement in a suitable language (C/Java/Python).
- Validate under varied loads for performance and reliability.



ENGINEER TO EXCEL

Module 1: Client Request Handling Module



Description:

- Accepts incoming client requests and organizes them for handling.

Functions:

- Accepts requests from multiple clients.
- Places each request into a queue.
- Preserves request order on a FIFO basis.
- Ensures no request is dropped.
- Manages multiple client connections at once.
- Readies queued requests for worker threads.

Purpose:

- To keep client requests flowing smoothly into processing.



ENGINEER TO EXCEL

Module 2: Thread Management Module



Description:

- Oversees creation and execution of worker threads for concurrent requests.

Functions:

- Builds and maintains a worker thread pool.
- Assigns each request to a free thread.
- Manages each thread's lifecycle from creation to termination.
- Uses system resources efficiently.
- Processes multiple requests at once.
- Balances load across threads for performance.

Purpose:

- To enable parallel request handling and boost performance.



ENGINEER TO EXCEL

Module 3: Concurrency Control Module



Description:

- Keeps shared-resource access safe and synchronized across threads.

Functions:

- Applies mutex and semaphore for synchronization.
- Prevents race conditions during execution.
- Avoids deadlocks for smooth thread execution.
- Regulates thread access to shared data.
- Preserves data consistency and integrity.
- Coordinates threads for safe resource sharing.

Purpose:

- To keep execution reliable and error-free.



ENGINEER TO EXCEL

Module 4: Response Generation Module



Description:

- Processes each request and builds the matching response.

Functions:

- Carries out the requested operation.
- Produces output tailored to the request.
- Delivers each response back to the requesting client.
- Keeps delivery timely.
- Manages multiple responses efficiently.
- Keeps server-client communication smooth.

Purpose:

- To deliver correct, timely responses for effective communication.

Module 5: Logging and Monitoring

Module

Description:

- Tracks server activity and monitors thread performance during execution.

Functions:

- Logs incoming requests and completed responses.
- Records thread status and resource usage.
- Flags errors and synchronization issues for review.
- Measures response time and throughput.
- Stores logs for later analysis and debugging.
- Supports real-time performance tracking.

Purpose:

- To provide visibility into server behavior and support system reliability.



ENGINEER TO EXCEL

Results and Recommendations



Results:

The thread-based simulation served multiple requests concurrently, giving faster responses and stronger performance than a single-threaded design. CPU use was efficient and the system stayed stable under load, while mutex and semaphore kept race conditions in check.

Recommendations:

- Adopt thread pooling to improve scalability.
- Apply load balancing for large-scale traffic.
- Refine synchronization to cut overhead.
- Add security mechanisms before deployment.
- Extend to real-time and dynamic applications.



ENGINEER TO EXCEL



Results and Recommendations



Concurrency Control Dashboard

Producer-Consumer thread pool · mutex · RW-lock · condition variables

PAUSED | t=28.8s

ARRIVAL RATE 12 REQ/S WORKER POOL 6 THREADS QUEUE CAPACITY 15 SLOTS SIM SPEED 1X

Play

Step

Reset

Burst

Deadlock

TASK MIX
STATIC 45% DB-READ 30% DB-WRITE 10% CPU 15%

Bounded Request Queue

13/15

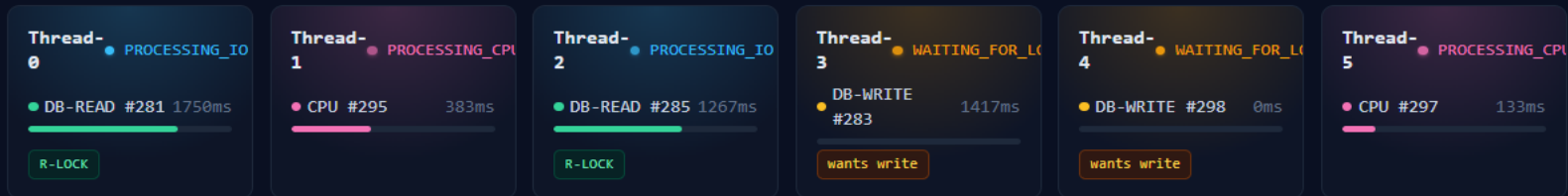


tail (enqueue) ←

head (dequeue) →

Worker Thread Pool

2 2 2



Synchronization Primitives



Live Telemetry

THROUGHPUT

7 rps

AVG LATENCY

2990 ms

P99 LATENCY

4850 ms

POOL UTIL

90 %

503 DROPS

95

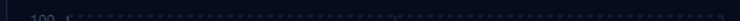
COMPLETED

209

THROUGHPUT & P99 (LAST 60S)



POOL UTILIZATION %





ENGINEER TO EXCEL

Reflection on Learning and Personal Development



- Gained hands-on knowledge of multithreading and concurrency control.
- Learned to service multiple requests at once.
- Deepened understanding of thread creation and management.
- Learned synchronization techniques like mutex and semaphore.
- Sharpened problem-solving through race conditions and deadlocks.
- Strengthened programming and logical thinking.
- Gained insight into real-world server efficiency.
- Sharpened time management and planning skills.
- Built confidence designing complex systems.



ENGINEER TO EXCEL

Conclusion



- A thread-based server handles multiple requests efficiently.
- Outperforms single-threaded designs.
- Cuts response time and improves user experience.
- Uses system resources efficiently.
- Relies on mutex and semaphore for synchronization.
- Prevents race conditions and deadlocks.
- Keeps execution safe and reliable.
- Shows why thread-based design matters today.



ENGINEER TO EXCEL

References



- Abraham Silberschatz, Peter B. Galvin, Greg Gagne – *Operating System Concepts*
- Andrew S. Tanenbaum – *Computer Networks*
- Java Multithreading Documentation
- POSIX Threads (Pthreads) Documentation
- Research papers on Web Server Architecture and Concurrency Control
- Online tutorials and technical blogs on multithreading and synchronization mechanisms

Thank You